WEB DEVELOPMENT

Learning Groups: P107, P108, E101, P102

Lecturer: Mr. Maina Samuel

Lesson 5 **learning topics**

- **5.0 java script history**
- **5.1 include JavaScript in html**
- **5.2 Output in java script**
- **5.3 java script variables**

5.0 java script introduction

The rise of the web has taken JavaScript places it was never conceived to be. It has become one of the most important languages in today's world. It was built to expand the web.

Origin of JavaScript

Netscape Communications had the vision that the web needed a way to become more **dynamic**. They wanted Animations, Interaction and other forms of small Automation as part of the **web** of the future. The goal was to seek ways to *expand the web*. And that is exactly what gave birth to JavaScript.

Brendan Eich, the father of **JavaScript**, was contracted by Netscape Communications to develop a scheme for the browser. The vision was to create something:

- *Easy to grasp syntactically*
- *Dynamic,*
- *To reduce verbosity,*
- *Speed up development,*
- *Powerful.*

Java was considered unsuitable for the type of audience that would consume **Mocha** (pre version of JavaScript) such as scripters, amateurs, designers as it took a lot of effort and time for such simple tasks. So the idea was to make Java available for big, professional, component writers, while Mocha would be used for small scripting tasks.

What is java script?

JavaScript (js) is a light-weight object-oriented programming language which is used by several websites for scripting the webpages.

Although, JavaScript has no connectivity with Java programming language. The name was suggested and provided in the times when Java was gaining popularity in the market. In addition to web browsers, databases such as Couch DB and MongoDB uses JavaScript as their scripting and query language.

JavaScript is a **client-side**, **object-based scripting language** that is used to handle and validate client-side data. JavaScript is also used for making the user interface of the web pages more dynamic making it responsive to events like the movement of the mouse, mouse click on a certain HTML element, a button clicks, etc, using which we can improve the user experience.

JavaScript is a **lightweight scripting language** based on **ECMAScript standards**. It executes on the browser and therefore reduces the load on the server. JavaScript can also be used to create cross-platform applications or SaaS applications or to create a Back-end server-side application by using **NodeJS**.

It is an **interpreted language** which means the script written inside Javascript is processed line by line and is not compiled before processing. These Scripts are interpreted by **JavaScript interpreter** which is a built-in component of the Web browser. Although this is changing lately, with the introduction of **V8 engine in Chrome** and NodeJS, now **just-in-time compilation** is also used for JavaScript.

JavaScript is P**latform Independent**, which means you need to write the script once and can run it on any platform or browser without affecting the output of the Script.

JavaScript, formerly known as LiveScript, has been developed by Netscape and Sun Microsystems. It is also known as ECMAScript as it was standardized by European Computer Manufacturer's Association(ECMA). The most recent upgrades were the **ECMAScript6**, **ES7**, **ES8**, **ES9**, **ES10** and the most recent **ECMAScript2020** which came at the beginning of this year. Although you should update yourself with all the

new features ECMAScript6 was the most significant change and we will be covering more about it towards the end of this tutorial series.

Browsers use their own JavaScript Engines to execute the JavaScript code. Some commonly used browsers are listed below:

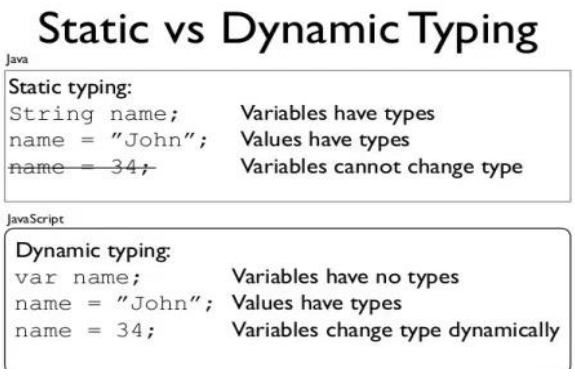
- Chrome uses a V8 engine.
- Firefox uses the SpiderMonkey engine.
- Microsoft Edge uses the ChakraCore engine.
- Safari uses the SquirrelFish engine.

Then comes JavaScript, which as we mentioned is a scripting language used for handling and validating client-side data, making webpages dynamic based on events, enabling features like cookies, local storage, etc and a lot more.

JavaScript Features

Following are some of the most useful features of JavaScript:

- **Light Weight** - *it is made for data handling at the browser level only. Since it is not a general-purpose language so it has a limited set of libraries.*
- **Dynamically Typed** -*JavaScript supports dynamic typing which means .js does not require the explicit declaration of the variables before they are used.*



java script uses implicit declaration

Code explanation

JS on the other hand is a dynamically typed language. Notice in the example above that when the variable 'name' is created, it wasn't declared with a data

type. This allows the 'name' variable to be dynamically changed. The example above shows that 'name' is assigned to "John," which is a String. Later, it can be assigned to 34, a Number. An error will not occur and JS will continue reading the code as normal.

- **Object-Based** –java script isn't a classed-based language- it's a prototype- based language **but** Starting from ES6, the concept of class and OOPs has been more refined. Also, in JavaScript, two important principles with OOP in JavaScript are Object Creation patterns (**Encapsulation**) and Code Reuse patterns (**Inheritance**). Although JavaScript developers rarely use this feature but its there for everyone to explore.
- **Functional** -functions in JavaScript can be used as objects and can be passed to other functions too.
- **Platform Independent**-you can write your JavaScript applications and run them on any platform or any browser without affecting the output of the Script.
- **Prototype-based**- in JavaScript a prototype is an object from which other objects inherit properties
- **Interpreted**- JavaScript is an interpreted language which means the script written inside JavaScript is processed line by line. These Scripts are interpreted by JavaScript interpreter which is a built-in component of the Web browser. But these days many JavaScript engines in browsers like the V8 engine in chrome uses just in time compilation for JavaScript code.
- **Asynchronous** -which means that the whole page will not have to wait for the JavaScript processing, and once the script processing completes we can easily modify the HTML code to show or hide data.

What can we build using JavaScript?

- **Websites:** JavaScript helps us to add behavior of our website. It helps users to interact with the website. For eg. clicking on buttons, saving details, uploading details on the website, etc.
- **Web Servers:** We can make robust server applications using JavaScript. To be precise we use JavaScript frameworks like Node.js and Express.js to build these servers.
- **Game Development:** In Game Development industry, JavaScript is used widely. With the addition of HTML5 Canvas, it's now possible to make 2D and 3D games in JavaScript very efficiently.

- **3D Drawings:** JavaScript in addition with HTML Canvas is used to make three-dimensional graphics.
- **Mobile Apps:** Mobile applications are the most popular modes of communicating these days. JavaScript also used to design mobile applications. There are many JavaScript frameworks using which we can make android, IOS, and hybrid apps.
- **Smartwatch Apps:** The popular smartwatch maker Pebble has created Pebble.js, a small JavaScript framework that allows a developer to create an application for the Pebble line of watches in JavaScript.

Advantages of JavaScript

JavaScript has various advantages that make it very useful as a scripting language, some are listed below:

1. JavaScript makes the webpage more interactive and dynamic.
2. By using JavaScript, you can make your webpage to give immediate feedback to the user so that they would not have to reload the page.
3. You can use JavaScript to **perform actions based on how the user interacts with your webpage** using the events generated like a mouse click, form submit, button click and a lot more.
4. JavaScript can save server traffic by **validating the user inputs** before even sending data to the server.
5. JavaScript can be used to **store client-side cookies** to store data on the client-side and then read them or delete them.
6. JavaScript can be used to **make async HTTP calls** to load data from the server.

Limitations of JavaScript

JavaScript has some limitations too which are listed below:

1. Like Java and other general-purpose programming languages, JavaScript is not capable to write **multi-threading or multiprocessor code**.
2. Core JavaScript can't be used for developing networking applications, although there are some frameworks available like NodeJS which can be used for this.
3. JavaScript can only be **used on the client-side** i.e. for frontend development.
4. JavaScript is processed in the browser, and **if someone disables JavaScript in their browser, no JavaScript code will be processed**. So if you are validating form values using JavaScript and if any user of your website switches JavaScript off in his/her browser, then your form validation will stop working.

5. JavaScript opens up a lot of doors and windows for security researchers and hackers to play around with a webpage.

JavaScript Frameworks

One major reason for the popularity of JavaScript is the JavaScript Frameworks. Here's a brief introduction of the most trending JavaScript frameworks:

*1. **AngularJS** is Google's web development framework which provides a set of modern development and design features for rapid application development.*

*2. **ReactJS** is another top JavaScript framework mainly maintained by Facebook and it's behind the User Interface of Facebook and Instagram, showing off its efficiency in maintaining such high traffic applications.*

*3. 3. **MeteorJS** is mainly used for providing back-end development. Using JavaScript on the back-end to save time and build expertise is one of the major ideas behind Meteor.*

*4. **jQuery** can be used when you want to extend your website and make it more interactive. Companies like Google, WordPress and IBM rely on jQuery. Etc...*

5.1 Include JavaScript in HTML

JavaScript provides 3 places to put the JavaScript code:

1. Between the head tag of html
2. Between the body tag of html
3. In .js file (external JavaScript)

1. Between the head tag of html

Let's put the **script** tag inside the HTML **head** element. The script placed inside the **head** element is loaded with the webpage and gets executed if any defined event occurs.

Edit This Code: See Result » Result:

```
<html>
<head>
    <title>my first JavaScript</title>
    <script>
    function text()
    {
        document.getElementById("script").innerHTML= "This text has been
written inside the JavaScript.";
    }
    </script>
</head>
<body>
    <p id="script">this is the old text</p>
    <button type="button" onclick=" text() ">click this button to use
JavaScript.</button>
</body>
</html>
```

This text has been written inside the JavaScript.

click this button to use JavaScript.

2. Between the body tag of html

The script tag inside the body element runs when a web page starts loading in a web browser. output

Edit This Code:

```
<html>
<body>
<script type="text/javascript">
 alert("Hello Javatpoint");
</script>
</body>
</html>
```



3. In .js file (external JavaScript)

- It provides **code re usability** because single JavaScript file can be used in several html pages.
- An external JavaScript file must be saved by .js extension. It is recommended to embed all JavaScript files into a single file. It increases the speed of the webpage.

Advantages of External JavaScript

There will be following benefits if a user creates an external javascript:

1. It helps in the reusability of code in more than one HTML file.
2. It allows easy code readability.
3. It is time-efficient as web browsers cache the external js files, which further reduces the page loading time.
4. It enables both web designers and coders to work with html and js files parallelly and separately, i.e., without facing any code conflictions.
5. The length of the code reduces as only we need to specify the location of the js file.

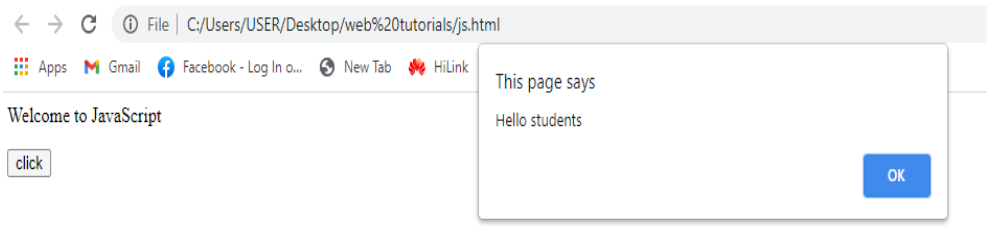


Save as index.html

```
function msg(){
 alert("Hello students");
}
```

Save as message.js

Output



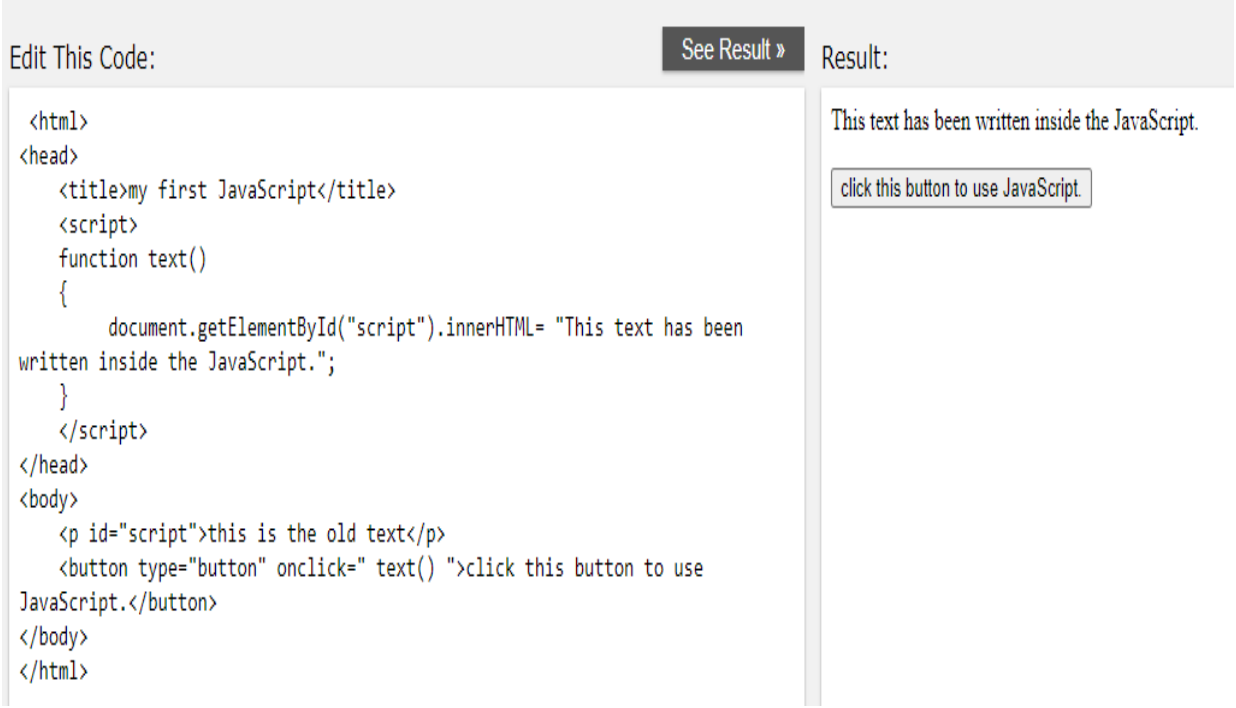
5.2 Output in JavaScript

We can get JavaScript Output in 4 simple and different ways on a webpage and these are given below. We can use them according to the application requirement. In this tutorial, we will be learning the following four different ways of getting output from JavaScript code, just like we use printf() in C language, cout in C++, etc.

1. Using innerHTML property
2. Using document. Write () method
3. Using **Alert Box**
4. By **logging on the Console**

JavaScript innerHTML Property

- The innerHTML property can be used along with getElementByid () within your JavaScript code to refer to an HTML element and **changes its content**
- HTML inner HTML is a property of the element that allows you to **get (**information about all DOM element**)** or **set** the HTML markup contained within the element

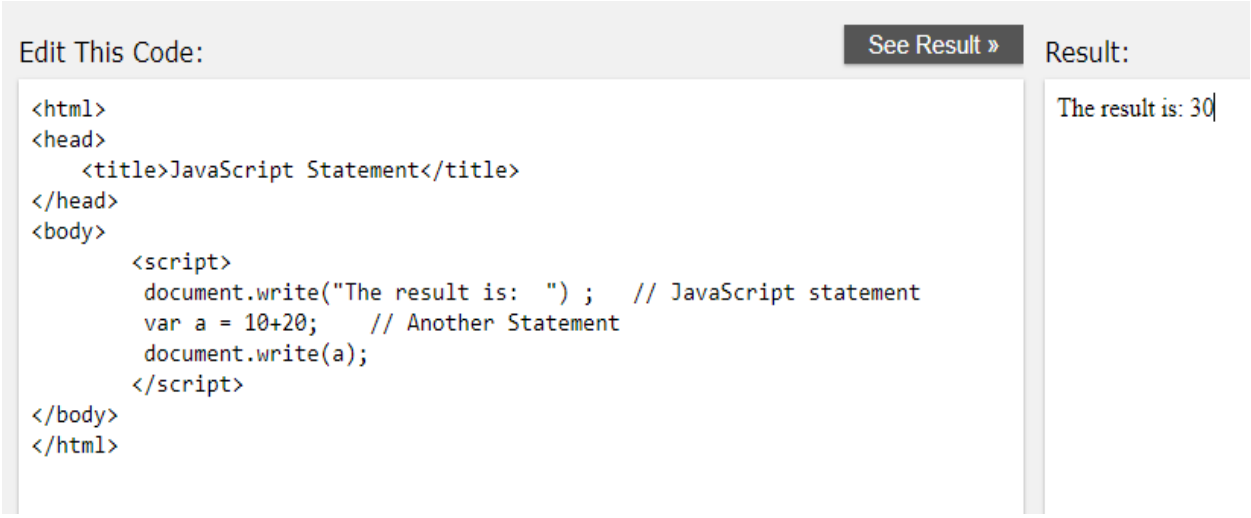


2. Using document. Write () method

JavaScript lets you write any output into the HTML webpage by using the document. Write () method. By using this **method**, you can directly write output to the HTML page.

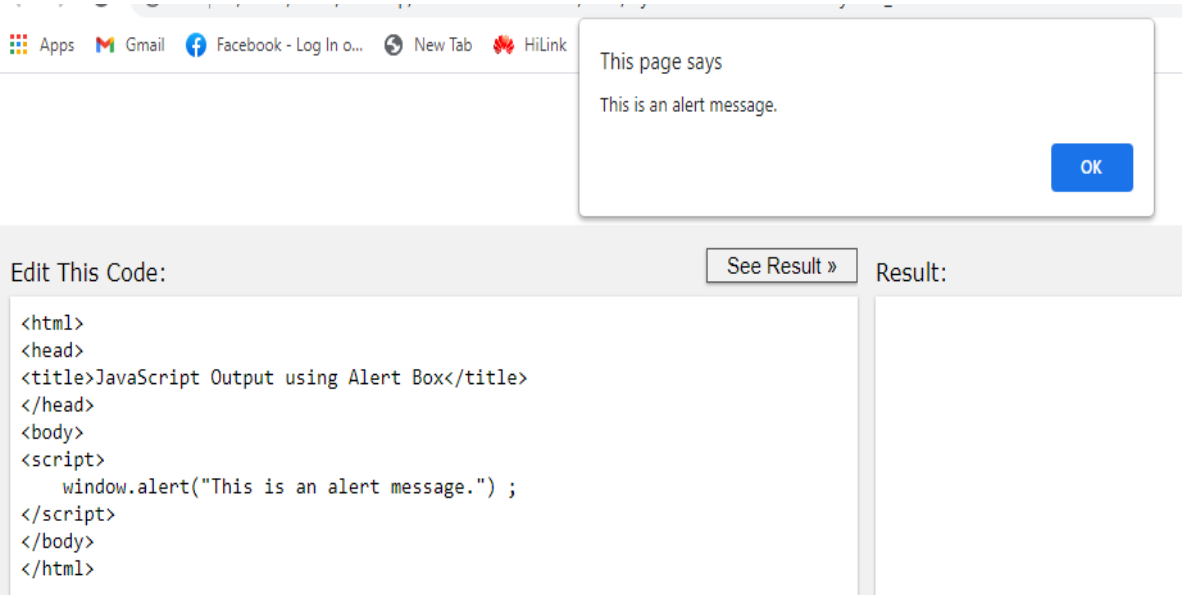
The write () method writes the HTML expressions or JavaScript code to a document. This method is mostly used for testing purposes.

document. Write () **this simply prints the specified text to the page.**



3. Using **Alert Box**

There are certain websites that give you **alert messages** when you access them or when you perform some action you see the **output message in alert boxes**. You can also make your webpage to send alert messages to notify something to the user using JavaScript, to use this feature you need to use the window. Alert () method.

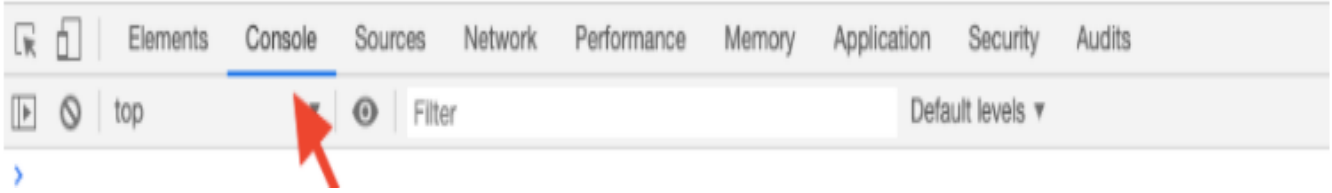


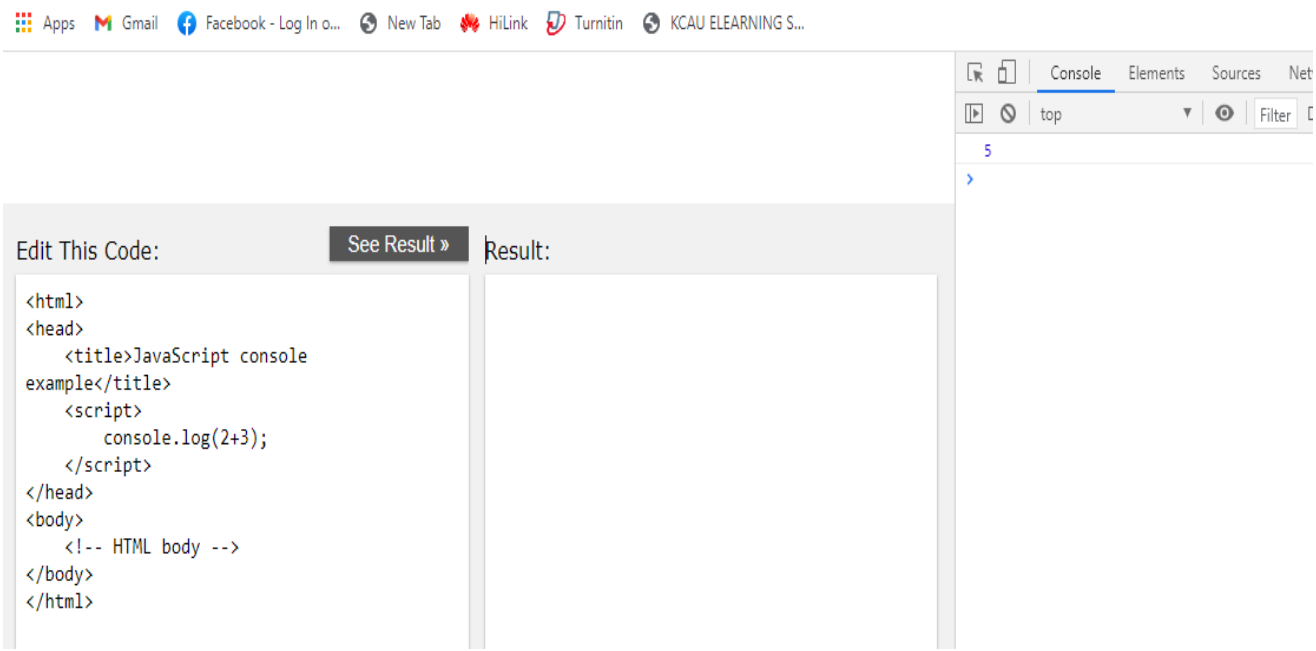
4. JavaScript Console Logging

JavaScript also lets you create console logs which can be seen in the **browser's developers' tools(Console)** for debugging purposes.

The statement written inside a console log will be executed but **would not be displayed in the browser instead it will be displayed inside the console of the browser**.

To open developer's tools in the Chrome browser, press **F12 in Windows**





5.3 java script variables

Variables are fundamental to all programming languages. Variables are used to store data, like string of text, numbers, etc.

You can create a variable with the **var** keyword, whereas the assignment operator (=) is used to assign value to a variable, like this: **var varName = value;**

NB: it's not recommended to declare a variable without **var** or **let** keyword **.it can accidently overwrite an existing global variable.** If define without **Var** keyword becomes **global variables.**

These are the following rules for naming a JavaScript variable:

- A variable name must start with a letter, underscore (_), or dollar sign ($).
- A variable name cannot start with a number.
- A variable name can only contain alpha-numeric characters (A-z, 0-9) and underscores.
- A variable name cannot contain spaces.
- A variable name cannot be a JavaScript keyword or a JavaScript reserved word.

```
<!DOCTYPE html>
<head>
     <title>Declaring Variables with let and const Keywords in
JavaScript</title>
</head>
<body>
     <script>
     // Declaring variables
     let name = "Harry Potter";
     let age = 11;
     let isStudent = true;

     // Printing variable values
     document.write(name + "<br>");
     document.write(age + "<br>");
     document.write(isStudent + "<br>");

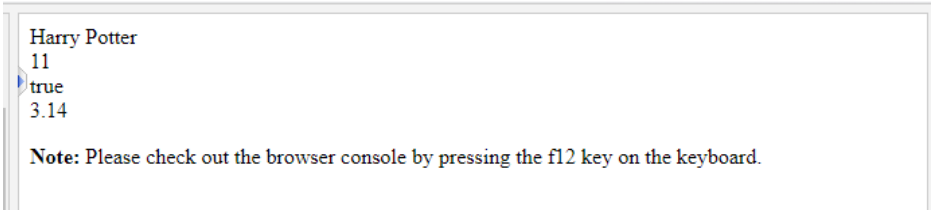
     // Declaring constant
     const PI = 3.14;

     // Printing constant value
     document.write(PI); // 3.14

     // Trying to reassign
     PI = 10; // error
     </script>

    <p><strong>Note:</strong> Please check out the browser console
by pressing the f12 key on the keyboard.</p>
</body>
</html>
```

output



The let and const Keywords ES6

ES6 introduces two new keywords let and const for declaring variables.

The const keyword works exactly the same as let, except that variables declared using const keyword cannot be reassigned later in the code. Here's an example:

What is the **let** and **var keyword** in JavaScript variable**?**

Both are used to declare the variable, but the **key difference** between **them lies in their scope.** var is **function scoped** while let is **block scoped**

Use let inside a code block:

JavaScript variable declared inside a **block** { } cannot be accessed from outside the block, if we have defined the variable using the let keyword. See the below example:

```
{
      let x = 2;
}

alert(x)   // not accessible
```

```
OUTPUT:
uncaught ReferenceError: x is not defined
```

the variable is **accessible only inside the block**. See the below example, to see that:

```
{
    let x = 2;
    alert(x)  // accessible
}
```

```
OUTPUT:2
```

JavaScript Local Variable

JavaScript Local variable is a variable that is **declared inside a code block or a function body or inside a loop body** and it has scope within the code block or the function. In simple words, the scope of a local variable is between the opening and closing curly braces { }, when declared and defined inside a code block or a function body.

Starting from **ES6** it is recommended to **use the let keyword** while declaring **local variables.**

Edit This Code: See Result » Result:

```
<html>
    <head>
        <script>
            function someFunc() {
                let num = 007; // local variable.
                document.writeln(num);
            }
            // calling the function
            someFunc();
            /*
                trying to access the variable
                outside its scope
            */
            document.writeln("/");
            document.writeln(num);
        </script>
    </head>
    <body>
        <!-- HTML body -->
    </body>
</html>
```

Result:

```
7 /
```

2. JavaScript Global Variable

JavaScript Global Variable is a variable that is **declared anywhere inside the script** and **has scope for the complete script execution**. Global variables are not declared inside any block or function but can be used in any function, or block of code.

Its recommended that we use the var keyword to declare the global variables, starting from ES6.

Edit This Code: See Result » Result:

```
<html>
    <head>
        <script>
            var num = 007; // global variable.

            function someFunc() {
                document.writeln(num);
            }
            // calling the function
            someFunc();
            /*
                trying to access the
                global variable
            */
            document.writeln("/");
            document.writeln(num);
        </script>
    </head>
    <body>
        <!-- HTML body -->
    </body>
</html>
```

Result:

```
7 / 7
```